

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение
высшего образования
**«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»**

Отчет
по лабораторной работе дисциплины
“Вычислительная математика”

Лабораторная работа №1
Вариант №10

Автор:
Надольский Кирилл Николаевич

Группа:
P3209

Преподаватели:
Рыбаков Степан Дмитриевич
Малышева Татьяна Алексеевна

Санкт-Петербург, 2026

Цель работы

Изучить численные методы решения СЛАУ и реализовать один из них средствами программирования.

Описание метода

Метод Гаусса-Зейделя является модификацией метода простой итерации и обеспечивает более быструю сходимость к решению систем уравнений.

Так же, как и в методе простых итераций строится эквивалентная СЛАУ и за начальное приближение принимается вектор правых частей (как правило, но может быть выбран и нулевой, и единичный вектор): $x^i_0 = (d_1, d_2, \dots, d_n)$.

Листинг программы

Ссылка на репозиторий:

https://github.com/kirnadols/itmo/blob/main/4%20%7C%20%D0%92%D1%8B%D1%87%D0%B8%D1%81%D0%BB%D0%B8%D1%82%D0%B5%D0%BB%D1%8C%D0%BD%D0%B0%D1%8F%20%D0%BC%D0%B0%D1%82%D0%B5%D0%BC%D0%B0%D1%82%D0%B8%D0%BA%D0%B0/lab1/lab1_calc.py

```
import sys
import os

def calculate_matrix_norm(A):
    n = len(A)
    norm = 0
    for i in range(n):
        if A[i][i] == 0:
            return float('inf')
        row_sum = sum(abs(A[i][j]) for j in range(n) if i != j) /
abs(A[i][i])
        if row_sum > norm:
            norm = row_sum
    return norm

def make_diagonally_dominant(A, b):
    n = len(A)
    new_A = [[0] * n for _ in range(n)]
    new_b = [0] * n
    used_rows = set()

    for i in range(n):
        row_found = False
        for r in range(n):
            if r in used_rows:
                continue

            diag_val = abs(A[r][i])
            sum_others = sum(abs(A[r][j]) for j in range(n) if j != i)

            if diag_val >= sum_others and A[r][i] != 0:
                new_A[i] = A[r][:]
                new_b[i] = b[r]
                used_rows.add(r)
                row_found = True
```

```

        break

    if not row_found:
        return A, b, False

return new_A, new_b, True

def gauss_seidel_method(A, b, epsilon, max_iterations=1000):
    n = len(A)

    A, b, is_dominant = make_diagonally_dominant(A, b)

    if not is_dominant:
        print("\n[ВНИМАНИЕ] Невозможно достичь строгого диагонального
преобладания.")
        print("Итерационный процесс может не сойтись!")
    else:
        print("\n[INFO] Матрица успешно приведена к диагональному
преобладанию.")

    for i in range(n):
        if A[i][i] == 0:
            raise ZeroDivisionError(
                f"Критическая ошибка: Диагональный элемент A[{i + 1}][{i +
1}] равен нулю. Деление на ноль невозможно.")

    norm = calculate_matrix_norm(A)
    print(f"[INFO] Норма матрицы C: {norm:.4f}")
    if norm >= 1:
        print("[ВНИМАНИЕ] Норма матрицы >= 1. Достаточное условие сходимости
не выполнено.")

    print("\n" + "-" * 40)
    print("ИТЕРАЦИОННЫЙ ПРОЦЕСС")
    print("-" * 40)

    x = [0.0] * n
    x_new = [0.0] * n
    iterations = 0
    errors = [0.0] * n

    formatted_start = ", ".join([f"{val:.4f}" for val in x])
    print(f"Итерация 0: x = [{formatted_start}]")

    for k in range(max_iterations):
        for i in range(n):
            s1 = sum(A[i][j] * x_new[j] for j in range(i))
            s2 = sum(A[i][j] * x[j] for j in range(i + 1, n))
            x_new[i] = (b[i] - s1 - s2) / A[i][i]

        errors = [abs(x_new[i] - x[i]) for i in range(n)]
        max_error = max(errors)
        iterations += 1

        formatted_x = ", ".join([f"{val:.4f}" for val in x_new])
        print(f"Итерация {iterations}: x = [{formatted_x}], max_error =
{max_error:.6e}")

        if max_error <= epsilon:
            print("\n[INFO] Достигнута заданная точность!")
            break

    x = x_new.copy()

```

```

    if iterations == max_iterations:
        print(
            f"\n[ВНИМАНИЕ] Достигнуто максимальное количество итераций
            ({max_iterations}). Точность {epsilon} не достигнута.")

    return x_new, iterations, errors

def read_from_file(filename):
    if not os.path.exists(filename):
        raise FileNotFoundError("Файл не найден. Проверьте правильность имени
и пути.")

    try:
        with open(filename, 'r', encoding='utf-8') as f:
            content = f.read()
    except Exception as e:
        raise IOError(f"Не удалось прочитать файл (возможно, нет прав
доступа): {e}")

    content = content.replace(',', '. ')
    tokens = content.split()

    if not tokens:
        raise ValueError("Файл абсолютно пуст.")

    try:
        n = int(tokens[0])
        epsilon = float(tokens[1])
    except ValueError:
        raise ValueError("Первые два значения в файле должны быть
размерностью (целое число) и точностью (число).")

    if n <= 0 or epsilon <= 0:
        raise ValueError("Размерность матрицы и точность в файле должны быть
строго больше 0.")
    if n > 21:
        raise ValueError("Размерность матрицы слишком велика. Проверьте
данные. (n ≤ 20)")

    expected_matrix_elements = n * (n + 1)

    if len(tokens) == 2 + expected_matrix_elements:
        max_iter = 1000
        matrix_tokens = tokens[2:]
    elif len(tokens) == 3 + expected_matrix_elements:
        try:
            max_iter = int(tokens[2])
            matrix_tokens = tokens[3:]
        except ValueError:
            raise ValueError("Третий параметр (максимальное число итераций)
должен быть целым числом.")
    else:
        raise ValueError(
            f"Несоответствие данных. Ожидалось {expected_matrix_elements}
коэффициентов матрицы, а в файле их {len(tokens) - 2}.")

    A = []
    b = []

    try:
        for i in range(n):
            start_idx = i * (n + 1)

```

```

        end_idx = start_idx + n
        row = [float(val) for val in matrix_tokens[start_idx:end_idx]]
        b_val = float(matrix_tokens[end_idx])
        A.append(row)
        b.append(b_val)
    except ValueError:
        raise ValueError("Один или несколько элементов матрицы в файле не являются числами.")

    return n, epsilon, max_iter, A, b

def get_input_mode():
    while True:
        mode = input("Ввести данные с клавиатуры (1) или из файла (2)? (Для выхода введите 0)\nВаш выбор: ").strip()
        if mode in ('0', '1', '2'):
            return mode
        print("\n[ОШИБКА] Степан Дмитриевич, Вас не понимаю. Введите только одну цифру: 1, 2 или 0.\n")

def get_valid_n():
    while True:
        try:
            user_input = input("Введите размерность матрицы n (от 1 до 20): ").strip()
            n = int(user_input)
            if 0 < n <= 20:
                return n
            else:
                print("\n[ОШИБКА] Размерность должна быть от 1 до 20 включительно.\n")
        except ValueError:
            print("\n[ОШИБКА] Это не целое число. Пожалуйста, введите цифрами (например, 3).\n")

def get_valid_epsilon():
    while True:
        try:
            user_input = input("Введите требуемую точность epsilon (например, 0.001): ").replace(',', '.').strip()
            epsilon = float(user_input)
            if epsilon > 0:
                return epsilon
            else:
                print("\n[ОШИБКА] Точность должна быть строго больше нуля.\n")
        except ValueError:
            print("\n[ОШИБКА] Это не число. Используйте точку или запятую для десятичных дробей.\n")

def get_valid_row(i, n):
    while True:
        try:
            row_input = input(f"Строка {i + 1} ({n + 1} чисел через пробел): ").replace(',', '.').strip().split()

            if len(row_input) != n + 1:
                print(f"\n[ОШИБКА] Вы ввели {len(row_input)} чисел, а нужно ровно {n + 1}. Попробуйте еще раз.\n")
                continue

```

```

        row_floats = list(map(float, row_input))
        return row_floats[:-1], row_floats[-1]
    except ValueError:
        print(f"\n[ОШИБКА] В строке обнаружен текст или недопустимые
символы. Вводите только числа через пробел!\n")

def get_valid_file_data():
    while True:
        filename = input("Введите имя файла (например, 19_19.txt) или '0' для
отмены: ").strip()
        if filename == '0':
            return None

        try:
            return read_from_file(filename)
        except Exception as e:
            print(f"\n[ОШИБКА ЧТЕНИЯ ФАЙЛА] {e}")
            print("Исправьте файл или укажите другой.\n")

def main():
    print("=" * 50)
    print("Решение СЛАУ методом Гаусса-Зейделя (Вариант 10)")
    print("=" * 50)

    try:
        while True:
            mode = get_input_mode()

            if mode == '0':
                print("Программа завершена. До свидания!")
                sys.exit(0)

            if mode == '2':
                data = get_valid_file_data()
                if data is None:
                    continue
                n, epsilon, max_iter, A, b = data
                print(f"\n[INFO] Данные из файла успешно загружены.")

            elif mode == '1':
                n = get_valid_n()
                epsilon = get_valid_epsilon()
                max_iter = 1000

                print(f"\nВведите коэффициенты матрицы A и вектор b
построчно.")
                print(f"Для размерности {n} каждая строка должна содержать
ровно {n + 1} чисел.")
                A = []
                b = []

                for i in range(n):
                    row_A, val_b = get_valid_row(i, n)
                    A.append(row_A)
                    b.append(val_b)

            try:
                x_ans, iters, errors = gauss_seidel_method(A, b, epsilon,
max_iter)

                print("\n" + "=" * 40)
                print("ИТОГОВЫЕ РЕЗУЛЬТАТЫ ВЫЧИСЛЕНИЙ")

```

```

print("=" * 40)
print("Вектор неизвестных x:")
for i in range(n):
    print(f"  x_{i + 1} = {x_ans[i]:.6f}")

print(f"\nОбщее количество итераций: {iters}")
print(f"\nВектор погрешностей |x_i^(k) - x_i^(k-1)| на
последнем шаге:")
for i in range(n):
    print(f"  e_{i + 1} = {errors[i]:.6e}")
print("\n")

except Exception as math_err:
    print(f"\n[МАТЕМАТИЧЕСКАЯ ОШИБКА] {math_err}")
    print("Попробуйте ввести другие данные.\n")

except KeyboardInterrupt:
    print("\n\nВыполнение прервано пользователем (Ctrl+C). До связи!")
    sys.exit(0)

if __name__ == "__main__":
    main()

```

Пример работы программы

Решение СЛАУ методом Гаусса-Зейделя (Вариант 10)
Ввести данные с клавиатуры (1) или из файла (2)? Введите 1 или 2: 1
Введите размерность матрицы n (от 1 до 20): 3
Введите требуемую точность epsilon: 0.01
Введите коэффициенты матрицы A и вектор b построчно:
Строка 1: 2 2 10 14
Строка 2: 10 1 1 12
Строка 3: 2 10 1 13

[INFO] Матрица успешно приведена к диагональному преобладанию.
[INFO] Норма матрицы C: 0.4000

ИТЕРАЦИОННЫЙ ПРОЦЕСС

Итерация 0: x = [0.0000, 0.0000, 0.0000]
Итерация 1: x = [1.2000, 1.0600, 0.9480], max_error = 1.200000e+00
Итерация 2: x = [0.9992, 1.0054, 0.9991], max_error = 2.008000e-01
Итерация 3: x = [0.9996, 1.0002, 1.0001], max_error = 5.179840e-03

[INFO] Достигнута заданная точность!

=====
ИТОГОВЫЕ РЕЗУЛЬТАТЫ ВЫЧИСЛЕНИЙ
=====

Вектор неизвестных x:
x_1 = 0.999555
x_2 = 1.000180
x_3 = 1.000053

Общее количество итераций: 3

Вектор погрешностей |x_i^(k) - x_i^(k-1)| на последнем шаге:
e_1 = 3.552000e-04

```
e_2 = 5.179840e-03  
e_3 = 9.649280e-04
```

```
Process finished with exit code 0
```

Вывод

В ходе выполнения данной лабораторной работы я познакомился с численными и итерационными методами решения СЛАУ, реализовав метод Гаусса-Зейделя на языке Python.